DEVIXO SOLUTIONS

Data Science Internship Program

Task 01 – Exploratory Data Analysis (EDA)

Telco Customer Churn

Student Name	Dua Sajid
Internship Domain	Data Science
Task Number	Task 01
Dataset	Telco Customer Churn (Kaggle, blastchar) – 7,043 customers, 21 columns
GitHub repository link	https://github.com/duasajid65-cloud/DevixoSolutions-Internship

Objective

Explore a real-world customer dataset with Python to find the patterns, relationships and business insights behind customer churn. The work covers data cleaning, feature analysis, correlation analysis, visualization and practical business recommendations.

Problem Statement

A telecom company is losing about one in four customers (26.5%). Before any prediction model is built, the company needs to know who is leaving, when they leave and what they have in common, so retention effort can be aimed at the right customers.

Dataset Description and Source

Source: Telco Customer Churn, Kaggle (author: blastchar)

Telco Customer Churn

Size: 7,043 customers and 21 columns

Column group	Columns
Customer profile	customerID, gender, SeniorCitizen, Partner, Dependents
Account	tenure, Contract, PaperlessBilling, PaymentMethod
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Charges	MonthlyCharges, TotalCharges
Target	Churn (Yes / No)

Technologies Used

Python, Pandas, NumPy, Matplotlib, Seaborn, Jupyter Notebook and GitHub.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Implementation Steps

1. Load the CSV with Pandas and explore its structure (Part 1).
2. Clean the data: missing values, duplicates, data types, and invalid values (Part 2).
3. Analyze numerical and categorical features, distributions, and outliers (Part 3).
4. Build the correlation matrix and heatmap (Part 4).
5. Create 15 visualizations (Part 5).
6. Write 15 business insights and 5 recommendations (Parts 6 and 7).

Data Import and Understanding (Part 1)

The data was loaded with pd.read_csv() and inspected with head(), tail(), shape, columns, info() and describe().

```
# ------------------------------
#       Import the Dataset
# ------------------------------
df = pd.read_csv("Telco-Customer-Churn.csv")
```
✓ 0.1s

```
# -----Display first 5 rows-----
df.head()
```
✓ 0.1s

Loading the dataset and displaying the first records

```
# -----Display last 5 rows-----
df.tail()
```
✓ 0.0s

Displaying the last records

```
# -----Check dataset shape-----
df.shape
```
✓ 0.0s

```
(7043, 21)
```

```
# -----Display column names-----
df.columns
```
✓ 0.0s

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='str')
```

Dataset shape (7,043 rows × 21 columns) and column names

```
    # -----Check data types-----
    df.info()
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
```

Data types and non-null counts from info()

```
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.9 MB
```

```
    # -----Identifying Numerical & Categorical columns-----
    print("Numerical:", df.select_dtypes(include = "number").columns.tolist())
    print("Categorical:", df.select_dtypes(exclude = "number").columns.tolist())
```

✓ 0.0s

```
Numerical: ['SeniorCitizen', 'tenure', 'MonthlyCharges']
Categorical: ['customerID', 'gender', 'Partner', 'Dependents', 'PhoneService', '
```

Numerical and categorical columns

```
# -----Basic Statistical Information-----
df.describe().round(2)
```
✓ 0.0s

	SeniorCitizen	tenure	MonthlyCharges
count	7043.00	7043.00	7043.00
mean	0.16	32.37	64.76
std	0.37	24.56	30.09
min	0.00	0.00	18.25
25%	0.00	9.00	35.50
50%	0.00	29.00	70.35
75%	0.00	55.00	89.85
max	1.00	72.00	118.75

Basic statistics from describe()

Before cleaning, three columns were stored as numeric (SeniorCitizen, tenure, MonthlyCharges), while TotalCharges was stored as text. After converting TotalCharges to numeric, four columns were available as numeric features for analysis.

Column	Mean	Min	Max
tenure (months)	32.37	0	72
MonthlyCharges	64.76	18.25	118.75
TotalCharges (after cleaning)	2,279.73	0.00	8,684.80

Data Cleaning Process (Part 2)

Missing values, duplicates, data types and invalid values were checked in that order.

```
# -----Check missing values-----
df.isnull().sum()
```

Checking missing values

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges       11
Churn               0
dtype: int64
```

Missing values before (left) and after (right) converting TotalCharges to numbers: 11 hidden blanks appear

```
# -----Check Total Charges
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors = "coerce")
df.isnull().sum()
```

Converting TotalCharges to numeric with errors="coerce"

```
# -----Handle missing values-----
df["TotalCharges"] = df["TotalCharges"].fillna(0)
```

✓ 0.0s

```
# -----Check & Remove duplicate records-----
print("Duplicate rows before removal:", df.duplicated().sum())
print("Duplicate customer IDs before removal:", df["customerID"].duplicated().sum())

df = df.drop_duplicates()

print("Duplicate rows after removal:", df.duplicated().sum())
print("Duplicate customer IDs after removal:", df["customerID"].duplicated().sum())
```

✓ 0.0s

```
Duplicate rows before removal: 0
Duplicate customer IDs before removal: 0
Duplicate rows after removal: 0
Duplicate customer IDs after removal: 0
```

Filling the 11 missing values with 0 and removing duplicates (none found)

```python
# -----Identify incorrect data types-----
print("---Data types after cleaning---\n")
print(df.dtypes)

print("\nTotalCharges data type:", df["TotalCharges"].dtype)
print("tenure data type:", df["tenure"].dtype)
print("MonthlyCharges data type:", df["MonthlyCharges"].dtype)
```

Checking data types

```
TotalCharges data type: float64
tenure data type: int64
MonthlyCharges data type: float64
```

TotalCharges is now float64

```python
# -----Check invalid or inconsistent values-----
print("Tenure range:", df["tenure"].min(), "to", df["tenure"].max())
print("Monthly Charges range:", df["MonthlyCharges"].min(), "to", df["MonthlyCharges"].max())
print("Total Charges range:", df["TotalCharges"].min(), "to", df["TotalCharges"].max())

print("\n-----Churn categories-----")
print(df["Churn"].unique())

print("\nInvalid tenure values:", (df["tenure"] < 0).sum())
print("Invalid monthly charges:", (df["MonthlyCharges"] < 0).sum())
print("Invalid total charges:", (df["TotalCharges"] < 0).sum())

print("\nUnique values in SeniorCitizen:", df["SeniorCitizen"].unique())
```

Checking for invalid or inconsistent values

```
Tenure range: 0 to 72
Monthly Charges range: 18.25 to 118.75
Total Charges range: 0.0 to 8684.8

-----Churn categories-----
<ArrowStringArray>
['No', 'Yes']
Length: 2, dtype: str

Invalid tenure values: 0
Invalid monthly charges: 0
Invalid total charges: 0

Unique values in SeniorCitizen: [0 1]
```

Value ranges are valid and there are no negative numbers

```python
# -----Verifying the dataset-----
print("Dataset shape:", df.shape)
print("Total missing values:", df.isnull().sum().sum())
print("Total duplicate rows:", df.duplicated().sum())
print("Duplicate customer IDs:", df["customerID"].duplicated().sum())

print("\n\n    -----Data types-----\n")
print(df.dtypes)
```

```
Dataset shape: (7043, 21)
Total missing values: 0
Total duplicate rows: 0
Duplicate customer IDs: 0
```

***Final verification**: no missing values and no duplicates.*

Check	Decision
Missing values	None at first glance, but TotalCharges held 11 blank spaces. Converting it to numbers turned them into 11 missing values.
Why filled with 0	All 11 customers have tenure 0, so they are brand new and have not been billed yet. 0 is the honest value.
Duplicates	No duplicate rows and no duplicate customerID.
Data types	TotalCharges changed from text to float. SeniorCitizen kept as 0/1 for correlation. Other types are correct.
Invalid values	tenure 0–72, MonthlyCharges 18.25–118.75, TotalCharges 0–8,684.80 and Churn only Yes/No. Nothing inconsistent.
Categories	"No internet service" and "No phone service" were kept as separate groups because they describe the customer's service status.

Feature Analysis (Part 3)

```python
# -----Numerical Features-----
numerical_columns = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

df[numerical_columns].describe()
```

✓ 0.0s

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

Numerical features: summary statistics

```
# -----Categorical Features-----
categorical_columns = df.select_dtypes(
    include=["object"]
).columns

categorical_columns
```

```
Index(['customerID', 'gender', 'Partner', 'Dependents', 'PhoneService',
       'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup',
       'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
       'Contract', 'PaperlessBilling', 'PaymentMethod', 'Churn'],
      dtype='str')
```

Categorical features

```
# -----Minimum & Maximum values-----
print("---Minimum values---\n", df[numerical_columns].min())
print("\n---Maximum values---\n", df[numerical_columns].max())
```

✓ 0.0s

```
---Minimum values---
 SeniorCitizen      0.00
tenure              0.00
MonthlyCharges     18.25
TotalCharges        0.00
dtype: float64

---Maximum values---
 SeniorCitizen        1.00
tenure               72.00
MonthlyCharges      118.75
TotalCharges       8684.80
dtype: float64
```

```
# -----Average values-----
df[numerical_columns].mean()
```

✓ 0.0s

```
SeniorCitizen        0.162147
tenure              32.371149
MonthlyCharges      64.761692
TotalCharges      2279.734304
dtype: float64
```

Minimum and maximum values (left) and average values (right)

```
Contract
Month-to-month    3875
Two year          1695
One year          1473
Name: count, dtype: int64
InternetService
Fiber optic    3096
DSL            2421
No             1526
Name: count, dtype: int64
PaymentMethod
Electronic check             2365
Mailed check                 1612
Bank transfer (automatic)    1544
Credit card (automatic)      1522
Name: count, dtype: int64
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

```
# -----Outliers (if any)----
for col in numerical_columns:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    low = q1 - 1.5 * iqr
    high = q3 + 1.5 * iqr
    count = ((df[col] < low) | (df[col] > high)).sum()
    print(col, 'Outliers:', count)
```

✓ 0.0s

```
SeniorCitizen Outliers: 1142
tenure Outliers: 0
MonthlyCharges Outliers: 0
TotalCharges Outliers: 0
```

Important categories (left) and IQR outlier check (right)

Numerical features. Customers stay 32 months on average and pay about $65 a month. The IQR test flags no outliers except SeniorCitizen, which is a 0/1 flag, so its 1,142 "outliers" are simply senior customers. Nothing was removed.

Categorical features. Month-to-month is the biggest contract group (3,875), fiber optic is the most common internet type (3,096), electronic check is the most used payment method (2,365) and 1,869 of 7,043 customers have churned.

Features showing the strongest observed differences in churn include contract type, tenure, internet service, payment method, online security and tech support. Gender and phone service make almost no difference (26.9% vs 26.2% and 24.9% vs 26.7%).

Feature	Highest churn group	Lowest churn group
Contract	Month-to-month 42.7%	Two year 2.8%
Internet service	Fiber optic 41.9%	No internet 7.4%
Payment method	Electronic check 45.3%	Credit card (auto) 15.2%
Tech support	No 41.6%	Yes 15.2%
Online security	No 41.8%	Yes 14.6%
Gender (weak feature)	Female 26.9%	Male 26.2%

Correlation Analysis (Part 4)

```
# -----Convert churn to numeric-----
df["Churn_num"] = df["Churn"].map({"Yes": 1, "No": 0})
print(df["Churn_num"].value_counts())
```

✓ 0.0s

```
Churn_num
0    5174
1    1869
Name: count, dtype: int64
```

Converting Churn to a number (Yes = 1, No = 0)

```
# -----Correlation Matrix & Heatmap-----
correlation = df[
    [
        "SeniorCitizen",
        "tenure",
        "MonthlyCharges",
        "TotalCharges",
        "Churn_num"
    ]
].corr()

correlation
```

✓ 0.0s

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn_num
SeniorCitizen	1.000000	0.016567	0.220173	0.103006	0.150889
tenure	0.016567	1.000000	0.247900	0.826178	-0.352229
MonthlyCharges	0.220173	0.247900	1.000000	0.651174	0.193356
TotalCharges	0.103006	0.826178	0.651174	1.000000	-0.198324
Churn_num	0.150889	-0.352229	0.193356	-0.198324	1.000000

Correlation matrix of the numerical features and churn

```
# -----Correlation Heatmap-----

plt.figure(figsize = (8, 5))
sns.heatmap(correlation, annot=True, cmap = "coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

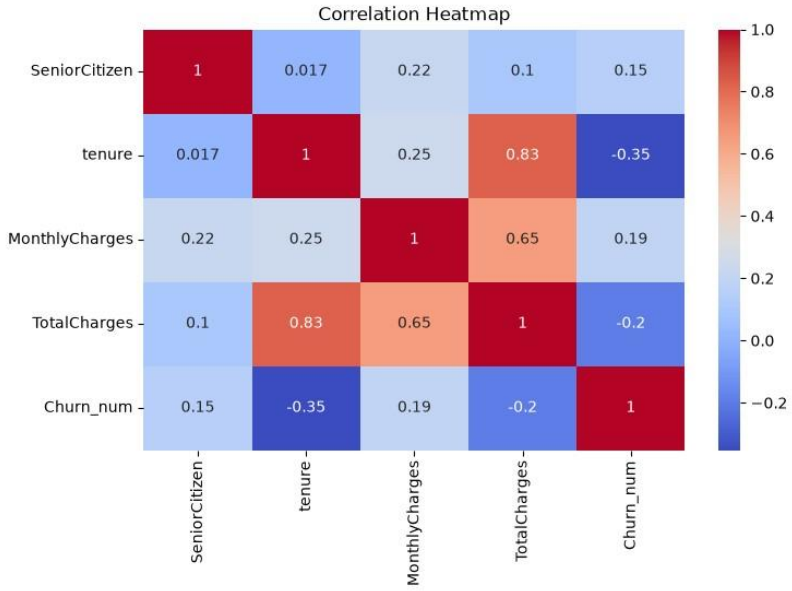


Figure 1: Correlation heatmap of numerical features and churn

Strong positive: tenure and TotalCharges (0.83). TotalCharges simply builds up over time, so it is not an independent driver of churn.

Moderate positive: MonthlyCharges and TotalCharges (0.65).

Strongest link with churn: tenure, at −0.35 (negative). Longer-staying customers churn less.

Weak: MonthlyCharges and churn (+0.19) and SeniorCitizen and churn (+0.15).

```
# -----Important Feature relationships-----
df["TenureGroup"] = pd.cut(df["tenure"], bins = [-1, 12, 24, 48, 60, 72],
                           labels = ["0-12", "13-24", "25-48", "49-60", "61-72"])
(df.groupby("TenureGroup", observed = True)["Churn_num"].mean() * 100).round(1)
```

```
TenureGroup
0-12     47.4
13-24    28.7
25-48    20.4
49-60    14.4
61-72     6.6
Name: Churn_num, dtype: float64
```

Churn rate (%) by tenure group: 47.4% in months 0–12 falling to 6.6% after month 60

Visualizations (Part 5)

```
# -----Histogram-----
plt.figure(figsize = (8, 5))
plt.hist(df["tenure"], bins = 20, color = "#3A6EA5", edgecolor = "white")
plt.title("Customer Tenure Distribution")
plt.xlabel("Tenure (Months)")
plt.ylabel("Number of Customers")
plt.show()

plt.figure(figsize = (8, 5))
sns.histplot(data = df, x = "MonthlyCharges", color = "#A9BCD0", kde = True)
plt.title("Distribution of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.ylabel("Number of Customers")
plt.show()
```

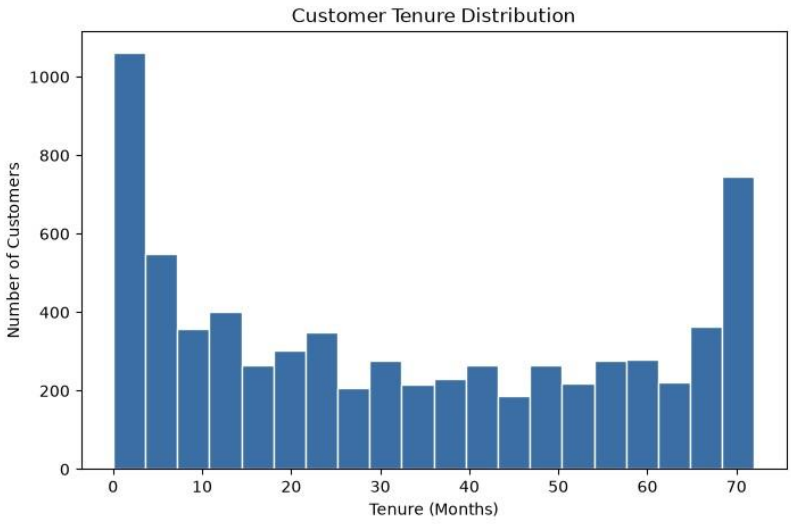


Figure 2: Customer tenure distribution (histogram)

Tenure is U-shaped. 1,481 customers are in their first 6 months and 1,407 have stayed over 60 months, with fewer in between.

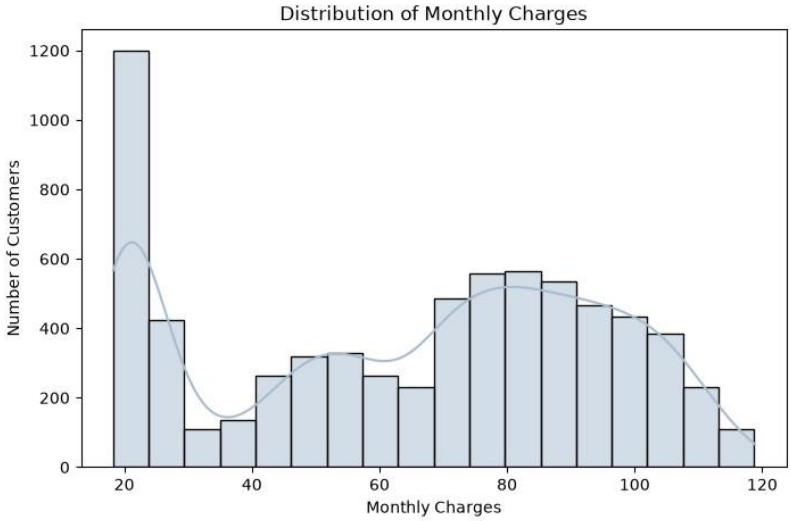


Figure 3: Monthly charges distribution (histogram)

A tall spike near $20 (1,389 customers below $25, mostly phone-only plans) and a wide spread between $60 and $100.

```
# -----Bar Chart-----
contract_churn = pd.crosstab(df["Contract"], df["Churn"], normalize = "index") * 100
contract_churn["Yes"].plot(kind = "bar", figsize = (10, 5), color = "#3A6EA5")
plt.title("Churn Rate by Contract Type")
plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation = 0)
plt.show()


payment_churn = pd.crosstab(df["PaymentMethod"],df["Churn"], normalize = "index") * 100
payment_churn["Yes"].sort_values(ascending = False).plot(kind = "bar", color = "#E4572E", figsize = (10, 5))
plt.title("Churn Rate by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation = 90, ha = "right")
plt.show()
```

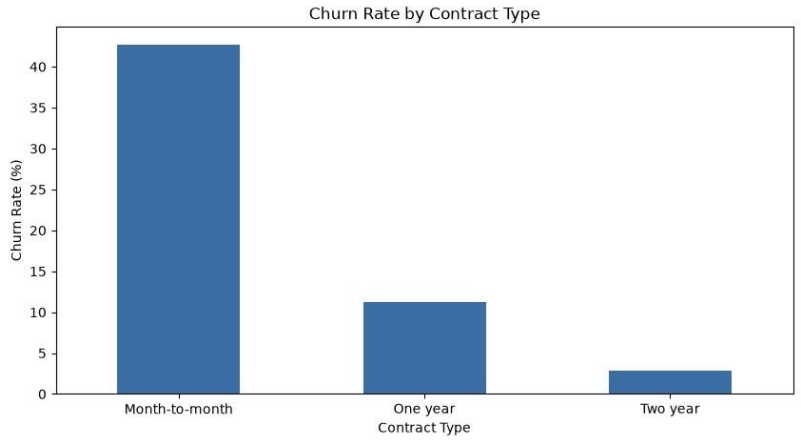


Figure 4: Churn rate by contract type (bar chart)

Month-to-month 42.7%, one year 11.3%, two year 2.8%. Contract length is the clearest split in the data.

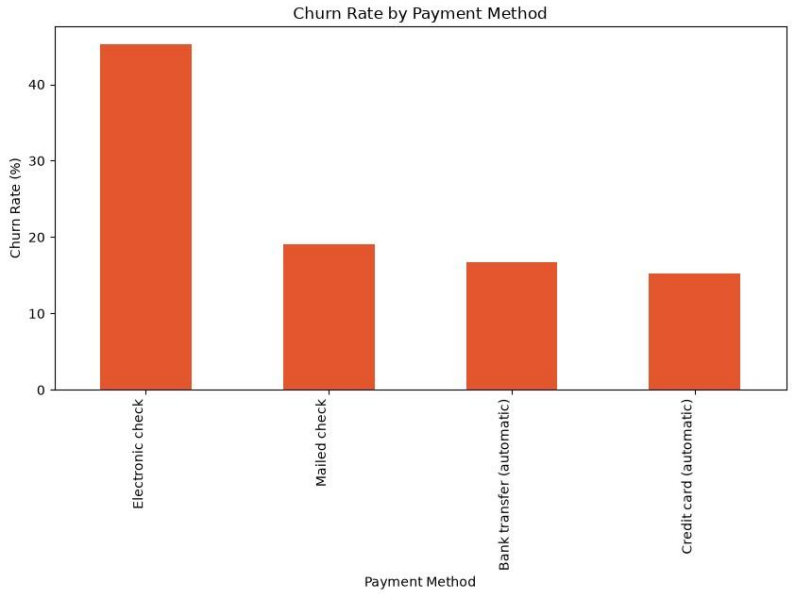


Figure 5: Churn rate by payment method (bar chart)

Electronic check customers churn at 45.3%. The other three methods sit between 15% and 19%.

```
# -----Count Plot-----
plt.figure(figsize = (8, 5))
sns.countplot(data = df, x = "Contract", hue = "Churn", palette = ["#3A6EA5", "#A9BCD0"])
plt.title("Customer Churn by Contract Type")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.show()

plt.figure(figsize = (8, 5))
sns.countplot(data = df, x = "InternetService", hue = "Churn", palette = ["#3A6EA5", "#A9BCD0"])
plt.title("Customer Churn by Internet Service")
plt.xlabel("Internet Service")
plt.ylabel("Number of Customers")
plt.show()
```

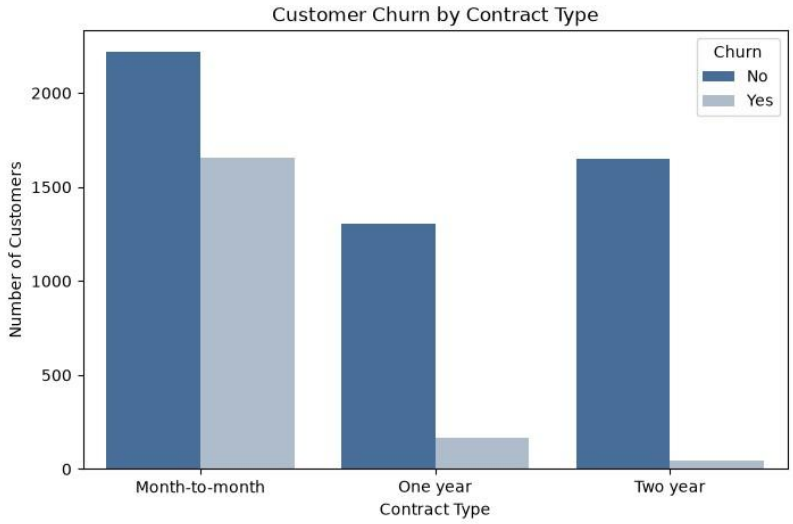


Figure 6: Churn by contract type (count plot)

1,655 of the 1,869 churners (88.6%) are on month-to-month contracts.

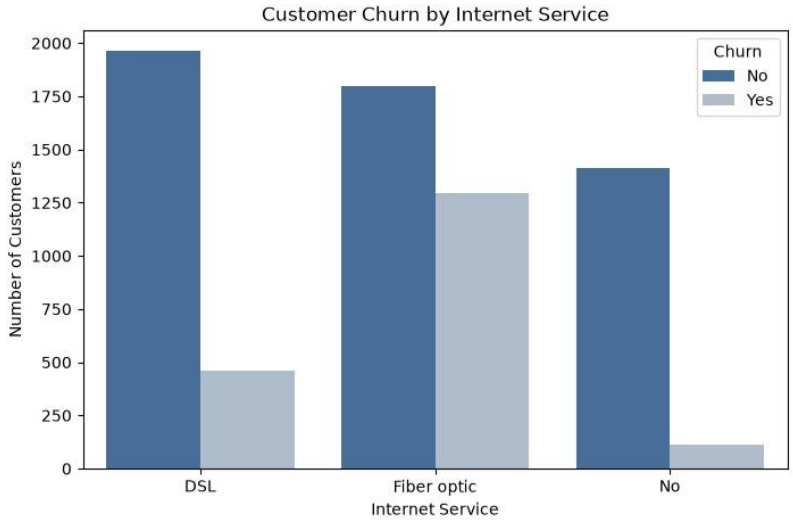


Figure 7: Churn by internet service (count plot)

Fiber optic has 1,297 churners against 459 for DSL, even though the two groups are of similar size.

```
# -----Pie Chart-----
churn_counts = df["Churn"].value_counts()
plt.figure(figsize = (6, 6))
plt.pie(churn_counts, labels = churn_counts.index, autopct = "%1.1f%%", startangle = 90, colors = ["#3A6EA5", "#A9BCD0"])
plt.title("Customer Churn Distribution")
plt.show()
```

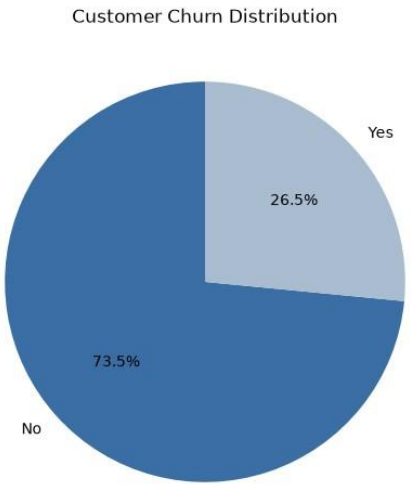


Figure 8: Overall churn split (pie chart)

73.5% stayed and 26.5% left, so the classes are imbalanced, which matters for any later model.

```
# -----Scatter Plot-----
plt.figure(figsize = (8, 5))
sns.scatterplot(data = df, x = "tenure", y = "MonthlyCharges", hue = "Churn", palette = ["#3A6EA5", "#E4572E"])
plt.title("Tenure vs Monthly Charges")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.show()
```

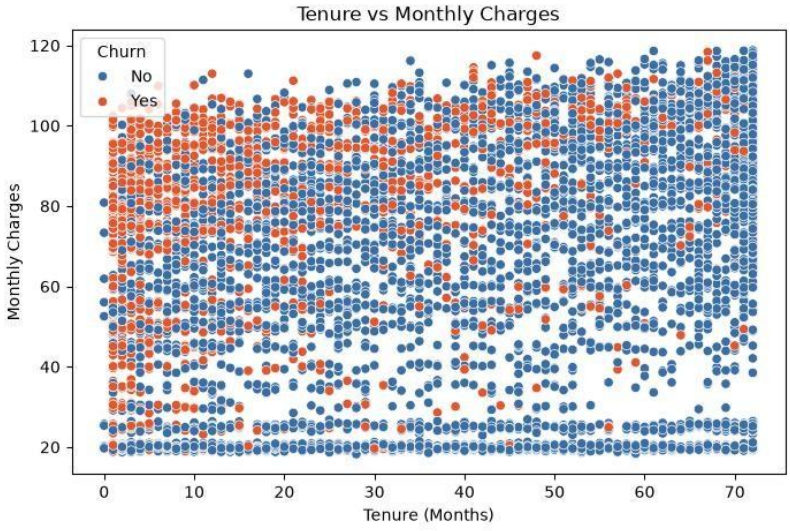


Figure 9: Tenure vs monthly charges (scatter plot)

Churned customers cluster at low tenure and at the higher charge levels (roughly $70 and above).

```
# -----Line Chart-----
df["TenureBand"] = pd.cut(df["tenure"],
    bins = [-1, 6, 12, 24, 36, 48, 60, 72],
    labels = [
        "0-6",
        "7-12",
        "13-24",
        "25-36",
        "37-48",
        "49-60",
        "61-72"
    ])
tenure_churn = pd.crosstab(df["TenureBand"], df["Churn"], normalize = "index") * 100
plt.figure(figsize = (9, 5))
plt.plot(tenure_churn.index.astype(str), tenure_churn["Yes"], marker = "o", color = "#3A6EA5")
plt.title("Churn Rate by Tenure Band")
plt.xlabel("Tenure Band (Months)")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation = 0)
plt.grid(True)
plt.show()
```

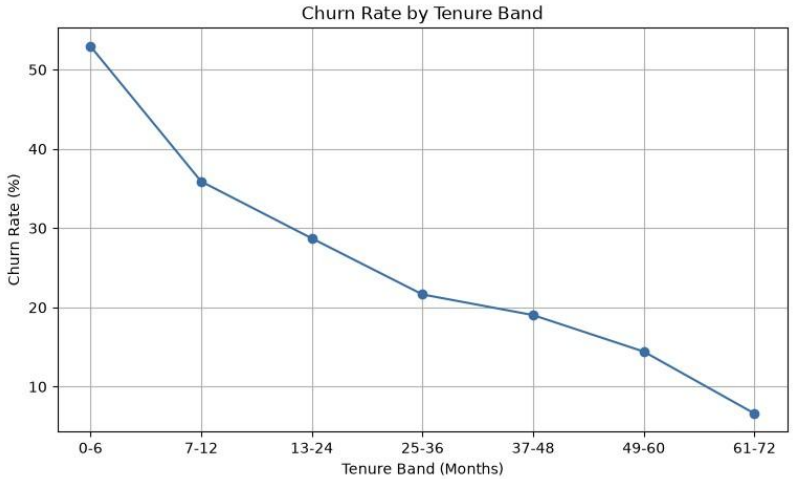


Figure 10: Churn rate by tenure band (line chart)

Churn falls steadily from about 53% in the first six months to 6.6% after month 60.

```
# -----Box Plot-----
plt.figure(figsize = (7, 5))
sns.boxplot(data = df,x = "Churn", y = "MonthlyCharges", color = "#A9BCD0")
plt.title("Monthly Charges by Churn Status")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")
plt.show()
```

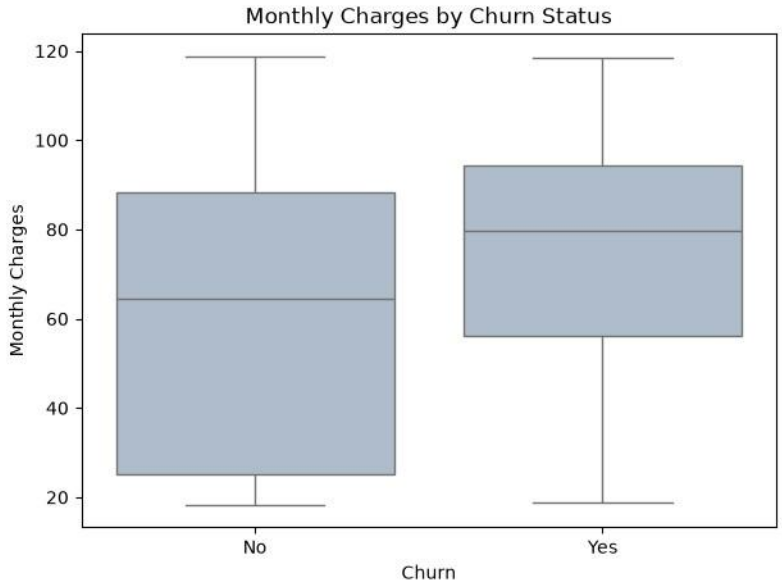


Figure 11: Monthly charges by churn status (box plot)

Median monthly charge is $79.65 for churners and $64.43 for customers who stayed.

```
# -----Violin Plot-----
plt.figure(figsize = (7, 5))
sns.violinplot(data = df, x = "Churn", y = "tenure", color = "#A9BCD0")
plt.title("Tenure Distribution by Churn Status")
plt.xlabel("Churn")
plt.ylabel("Tenure (Months)")
plt.show()
```

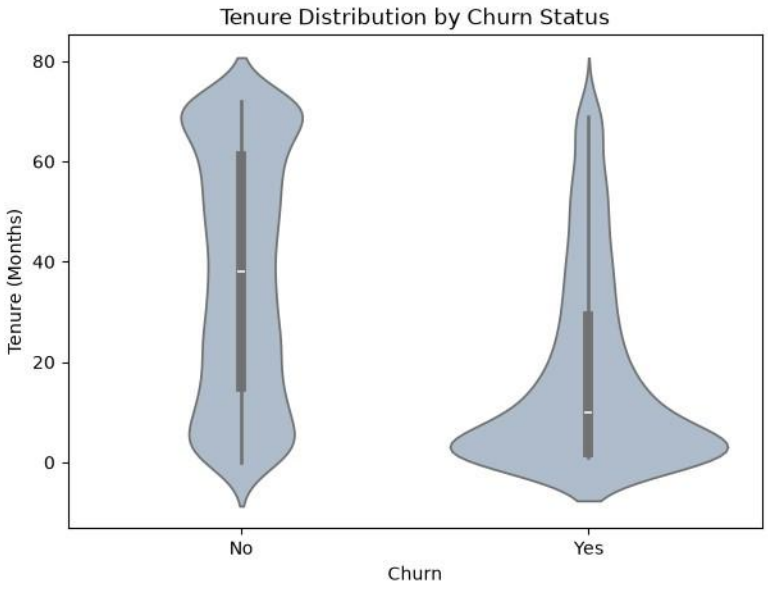


Figure 12: Tenure by churn status (violin plot)

Churners are concentrated at very short tenures (median 10 months). Retained customers are spread out with a median of 38 months.

```
# -----Pair Plot-----
sns.pairplot(df[["tenure", "MonthlyCharges", "TotalCharges", "Churn"]], hue = "Churn", palette = ["#3A6EA5", "#E4572E"])
plt.show()
```

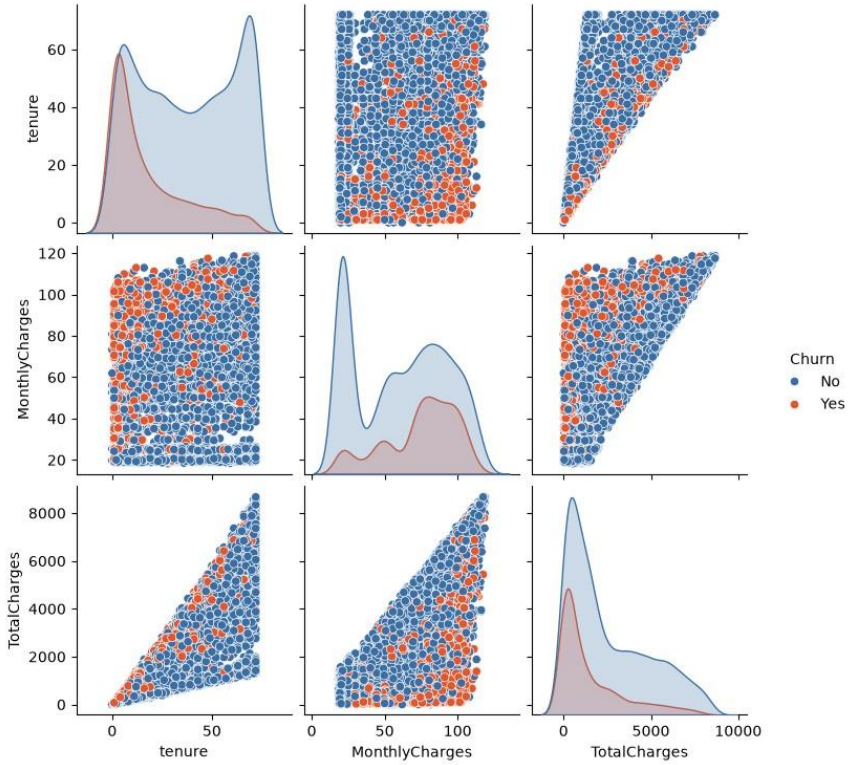


Figure 13: Pair plot of numerical features

Confirms the tenure–TotalCharges link and shows churned customers bunched in the low-tenure, low-total area.

```
# -----Distribution Plot-----
plt.figure(figsize = (12, 6))
sns.kdeplot(data = df, x = 'MonthlyCharges', hue = 'Churn', fill = True, palette = ["#3A6EA5", "#E4572E"])
plt.title('Monthly Charges Distribution by Churn')
plt.show()
```

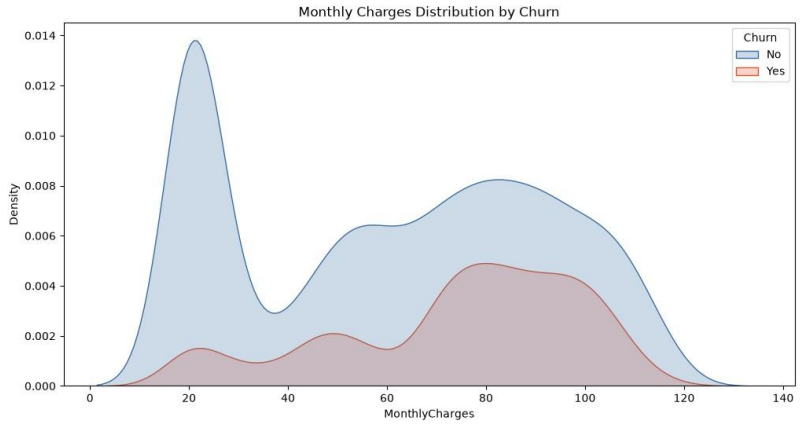


Figure 14: Monthly charges distribution by churn (distribution plot)

The churn curve peaks around $70–$100 while the retained curve has its main peak near $20–$30.

```
# -----Area Chart-----
tenure_counts = pd.crosstab(df["TenureBand"], df["Churn"])
tenure_counts.plot(kind = "area", figsize = (10, 6), alpha = 0.6, color = ["#3A6EA5", "#E4572E"])
plt.title("Customer Distribution by Tenure Band")
plt.xlabel("Tenure Band (Months)")
plt.ylabel("Number of Customers")
plt.xticks(rotation = 0)
plt.legend(title = "Churn")
plt.show()
```

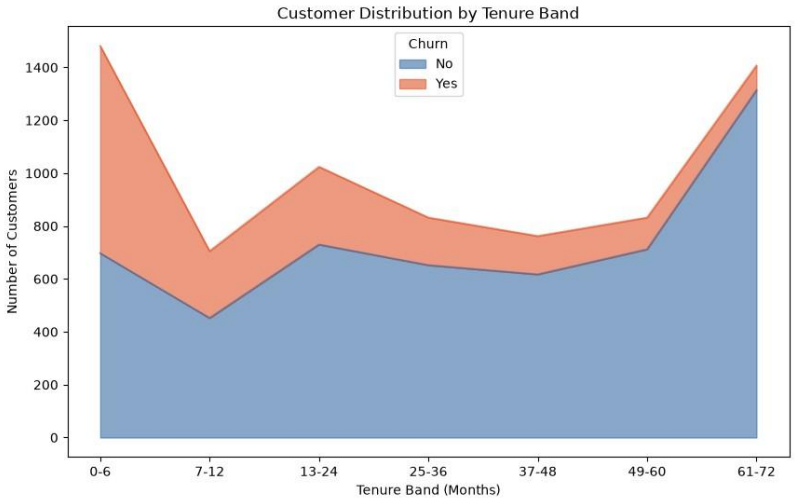


Figure 15: Customers by tenure band (area chart)

Retained customers pile up in the first and last bands. Churned customers are mostly in the early bands.

Business Insights (Part 6)

1. **Overall churn is high:** 1,869 of 7,043 customers (26.5%) have left, roughly one in four.

2. **Contract type is the clearest dividing line:** Churn is 42.7% on month-to-month, 11.3% on one-year, and only 2.8% on two-year contracts.

3. **Month-to-month customers make up most churners:** They are 55% of customers but 88.6% of churners (1,655 of 1,869).

4. **Churn decreases sharply with tenure**: it is 47.4% across the 0–12 month group and falls to 6.6% after month 60. Within the first six months, the churn rate is about 53%. Churners have a median tenure of 10 months against 38 for stayers, and tenure has the strongest correlation with churn (−0.35).

5. **The first year is the danger zone:** Customers in their first 12 months are 31% of the base but 55.5% of all churners (1,037).

6. **Churn takes a large share of revenue:** Churned customers paid $139,131 a month, which is 30.5% of all monthly charges ($456,117).

7. **Fiber optic is the highest-churn internet service category:** Fiber churn is 41.9% against 19.0% for DSL and 7.4% for no internet, and fiber accounts for 69.4% of all churners.

8. **Fiber customers have higher average monthly charges**: Average monthly charge is $91.50 for fiber against $58.10 for DSL, so the relationship between price and perceived value is worth checking.

9. **Electronic check is a risk signal:** 45.3% of electronic check customers churn (1,071 of 2,365), against 15–19% for the other three payment methods.

10. **Paperless billing goes with higher churn:** 33.6% churn with paperless billing against 16.3% without, though this overlaps with electronic check use.

11. **Missing protection services goes with churn:** Customers without online security churn at 41.8% (14.6% with it) and without tech support at 41.6% (15.2% with it).

12. **Tech support matters most for fiber:** Fiber customers without tech support churn at 49.4% against 22.6% with it.

13. **Household ties help retention:** Customers without a partner churn at 33.0% (19.7% with one) and without dependents at 31.3% (15.5% with them).

14. **Senior citizens are a high-risk group:** They are 16.2% of customers but churn at 41.7% against 23.6% for non-seniors.

15. **Higher bills go with higher churn, but only weakly:** Churners pay a mean of $74.44 against $61.27 (median $79.65 vs $64.43). Churn is 10.9% for bills up to $35 and 35.4% above $70, but the correlation is only 0.19. Gender and phone service show almost no effect.

Business Recommendations (Part 7)

1. **Move month-to-month customers onto longer contracts.** Month-to-month churn is 42.7% against 2.8% on two-year plans, and 1,655 churners came from this group. Offer a small discount or a free add-on for switching to a one- or two-year plan. Average bills are similar across contracts ($66.40 vs $60.77), so a modest incentive is realistic.

2. **Build a first-year retention programme.** 47.4% of customers in months 0–12 leave, and about 53% in the first six months. Welcome calls and check-ins at months 1, 3 and 6, plus an early upgrade offer, would reach the 55.5% of churners who go in year one.

3. **Review fiber optic pricing and service quality.** Fiber churn is 41.9% with average bills of $91.50, and fiber supplies 69.4% of churners. Survey fiber customers on speed, outages and price, and test a price-matched plan.

4. **Push electronic check customers to automatic payment.** They churn at 45.3% against about 15% for card auto-pay, and there are 2,365 of them. Offer a one-time bill credit for switching to bank or card auto-pay.

5. **Bundle online security and tech support for at-risk customers.** The 2,553 customers with neither service churn at 49.0%, and fiber customers without tech support churn at 49.4% against 22.6% with it. Include a free trial of both in new fiber and month-to-month plans.

Priority target: month-to-month, fiber, electronic check customers. This overlap is 1,307 customers with a 60.4% churn rate, the best place to start.

Challenges Faced

- TotalCharges looked clean but was stored as text with 11 hidden blanks. Checking tenure showed they were new customers, so 0 was the right fill.
- Many columns use "No internet service" or "No phone service" as a separate label, which needs care when comparing groups.
- The IQR outlier test flagged the 0/1 SeniorCitizen column, which is not a real outlier problem.
- Choosing charts that say something useful rather than just reaching the 12-chart minimum.
- Several factors overlap (paperless billing with electronic check, fiber with higher bills), so single-factor results had to be read carefully.

Conclusion

The analysis of 7,043 Telco customers shows that churn is closely tied to contract type and tenure. Month-to-month customers and first-year customers account for most departures. Fiber optic service, electronic check payment, missing security and support services, and higher monthly bills also stand out. Tenure has the strongest relationship with churn among the numerical variables (−0.35).

These findings point to clear retention targets: new customers, month-to-month customers, and fiber customers without support. The results are associations from EDA and not proof of cause so the next step would be a predictive model with feature engineering.

GitHub Repository Link

https://github.com/duasajid65-cloud/DevixoSolutions-Internship